Quiz 2 Solution All Variants

Variant 1

Question 1: Huffman Encoding (5 marks)

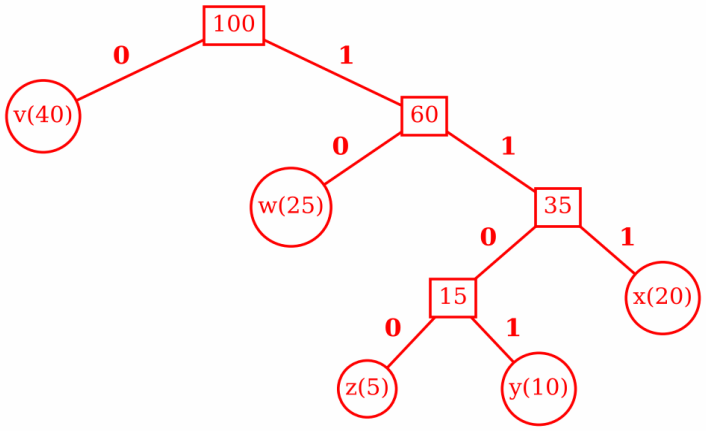
Given frequencies (100 characters): v = 40, w = 25, x = 20, y = 10, z = 5

(a) Merge Steps [2 marks]

1. z(5) + y(10) → 15
2. 15 + x(20) → 35
3. w(25) + 35 → 60
4. v(40) + 60 → 100 (root)

(b) Huffman Tree and Codes [1 mark]

Smaller-weight child on the left (edge 0), larger-weight child on the right (edge 1).



Symbol	Frequency	Code
v	40	0
w	25	10
x	20	111
z	5	1100
y	10	1101

(c) Total Bits Comparison [1 mark]

Huffman: 40(1) + 25(2) + 20(3) + 10(4) + 5(4) = 40 + 50 + 60 + 40 + 20 = **210 bits**

Fixed-length (3 bits): 100 × 3 = **300 bits**

Savings: 300 − 210 = **90 bits**

(d) Encode the String zvxw [1 mark]

z = 1100, v = 0, x = 111, w = 10

Encoded string: 1100 0 111 10 = **1100011110**

Question 2: Activity Selection (5 marks)

Activity	**A1**	**A2**	**A3**	**A4**	**A5**	**A6**	**A7**	**A8**
Start	1	2	3	5	6	7	9	10
Finish	3	4	6	7	8	10	11	12

(a) Earliest-Finish-Time Greedy Algorithm [3 marks]

Activities are already sorted by finish time.

Activity	**Decision**	**Reason**
A1 (1,3)	Accept	First activity; last finish = 3
A2 (2,4)	Reject	2 < 3
A3 (3,6)	Accept	3 ≥ 3; last finish = 6
A4 (5,7)	Reject	5 < 6
A5 (6,8)	Accept	6 ≥ 6; last finish = 8
A6 (7,10)	Reject	7 < 8
A7 (9,11)	Accept	9 ≥ 8; last finish = 11
A8 (10,12)	Reject	10 < 11

Selected set: {A1, A3, A5, A7}

Size: 4 activities

(b) Add Activity N = (4, 5) [1 mark]

New finish-time order: A1, A2, N, A3, A4, A5, A6, A7, A8

Activity	**Decision**	**Reason**
A1 (1,3)	Accept	First activity; last finish = 3
A2 (2,4)	Reject	2 < 3
N (4,5)	Accept	4 ≥ 3; last finish = 5
A3 (3,6)	Reject	3 < 5
A4 (5,7)	Accept	5 ≥ 5; last finish = 7
A5 (6,8)	Reject	6 < 7
A6 (7,10)	Accept	7 ≥ 7; last finish = 10
A7 (9,11)	Reject	9 < 10
A8 (10,12)	Accept	10 ≥ 10; last finish = 12

New selected set: {A1, N, A4, A6, A8}

Size: 5 activities

(c) Explanation [1 mark]

Choosing the activity that finishes earliest leaves the maximum remaining time, so the largest number of compatible activities can still be selected.

Variant 2

Question 1: Counting Sort (5 marks)

Given: A = (2,p), (0,q), (2,r), (1,s), (0,t), (2,u), (1,v) , keys in range 0..2 (k = 2)

(a) Count Array C [2 marks]

After counting: key 0 appears 2 times, key 1 appears 2 times, key 2 appears 3 times.

C = [2, 2, 3]

After prefix sum: C[0] = 2, C[1] = 2 + 2 = 4, C[2] = 4 + 3 = 7

C = [2, 4, 7]

(b) Output Array B [2 marks]

Scan A from right to left; place each element at position C[key], then decrement C[key].

Element	Position	Updated C
(1,v)	4	[2, 3, 7]
(2,u)	7	[2, 3, 6]
(0,t)	2	[1, 3, 6]
(1,s)	3	[1, 2, 6]
(2,r)	6	[1, 2, 5]
(0,q)	1	[0, 2, 5]
(2,p)	5	[0, 2, 4]

B = (0,q), (0,t), (1,s), (1,v), (2,p), (2,r), (2,u)

(c) Show that the Sort is Stable [1 mark]

Elements with equal keys keep their original relative order from the input:

- Key 0: q before t (input and output)
- Key 1: s before v (input and output)
- Key 2: p, r, u (input and output)

Hence the sort is stable.

Question 2: Linear-Time Sorting (5 marks)

(a) Comparison-Sort Lower Bound [2 marks]

The Ω(n log n) lower bound applies only to comparison-based sorts, which decide order by comparing pairs of elements. Counting sort does not compare elements; it uses the key values directly as array indices. So it is not bound by that limit and runs in Θ(n + k) time without contradicting it.

(b) Choose the Most Suitable Algorithm [3 marks]

i. n = 10 integers in the range 0..999

- **Algorithm:** Counting sort
- **Time:** Θ(n + k) = Θ(10 + 1000) = Θ(n)
- **Reason:** The key range (k = 1000) is small compared to n.

ii. n = 10 student IDs, each with 8 decimal digits

- **Algorithm:** Radix sort
- **Time:** Θ(d(n + b)) = Θ(8(10 + 10)) = Θ(n)
- **Reason:** Counting sort would need an array of size 10 ; radix sort sorts digit by digit in 8 passes instead.

iii. n = 10 sensor readings, uniformly distributed in [0, 100)

- **Algorithm:** Bucket sort
- **Expected time:** Θ(n)
- **Reason:** Uniform distribution spreads readings evenly across n buckets, so each bucket holds O(1) elements on average.

Variant 3

Question 1: Greedy Algorithm for Maximum Exam Score (4 marks)

Will prioritizing problems with maximum marks always give an optimal solution?

No. Marks are equally divided among checkpoints, so partial attempts earn partial marks (fractional knapsack). A high-mark problem may take a long time; the right greedy choice is marks per minute, not total marks.

Counterexample

- Total time T = 30 minutes
- Number of questions N = 2
- Marks M = [100, 90]
- Expected time E_Time = [30, 10]

Greedy Approach (Highest Marks First)

- Select Q1 (100 marks, 30 minutes); no time left.
- Total score = **100 marks**

Better Approach (Marks per Minute)

- Q1: 100/30 = 3.33 marks/min; Q2: 90/10 = 9 marks/min

- Attempt Q2 first: 90 marks in 10 minutes.
- Spend the remaining 20 minutes on Q1: (20/30) × 100 = 66.67 marks.
- Total score = 90 + 66.67 = **156.67 marks**

Conclusion: Greedy by maximum marks gives 100, while the optimal score is 156.67, so the approach is not optimal. Sorting by marks/time ratio gives the optimal score.

Question 2: Coin Denomination Problem (6 marks)

Task 1: Implement the Greedy Algorithm [3 marks]

Algorithm: Always choose the largest coin that does not exceed the remaining amount.

```
def greedy_coin_change(N, denominations):
    denominations = sorted(denominations, reverse=True)
    coins_used = []

    for coin in denominations:
        while N >= coin:
            N -= coin
            coins_used.append(coin)

    if N != 0:
        return None   # Amount cannot be formed

    return coins_used

coins = greedy_coin_change(27, [1, 2, 5, 10])
print("Minimum coins required =", len(coins))
print("Coins used =", coins)
```

Output: Minimum coins required = 4, Coins used = [10, 10, 5, 2]

Task 2: Test Case Where Greedy Fails [2 marks]

Input: N = 6, Denominations = [1, 3, 4]

- **Greedy:** choose 4 (remaining 2), then 1 and 1 → [4, 1, 1] = 3 coins
- **Optimal:** [3, 3] = 2 coins

Greedy uses 3 coins instead of the optimal 2, so it fails.

Task 3: Approach That Always Finds the Optimal Solution [1 mark]

Use **Dynamic Programming**, which considers all coin choices.

- dp[0] = 0, and dp[i] = ∞ initially for i ≥ 1
- dp[i] = 1 + min(dp[i − c]) over all coins c ≤ i
- The answer is dp[N] (minimum coins for amount N), in O(N × number of coins) time.

Variant 4

Question 1: Huffman Encoding (5 marks)

Symbol	a	b	c	d	e	f

Frequency	8	3	27	12	15	35

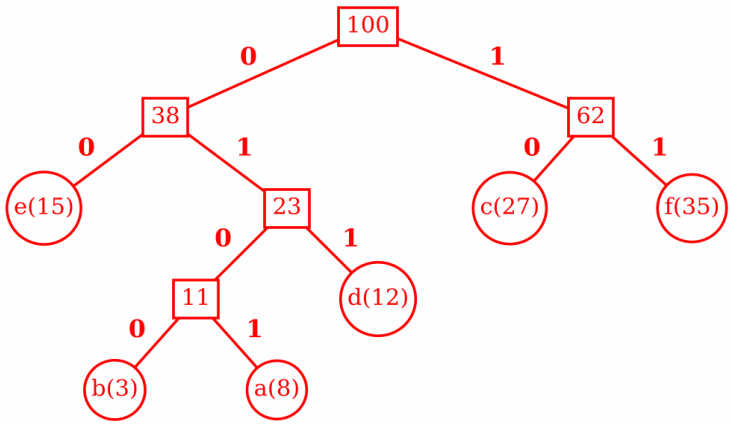
(a) Merge Steps [2 marks]

Initial nodes sorted by weight: b(3), a(8), d(12), e(15), c(27), f(35)

1. b(3) + a(8) → 11
2. 11 + d(12) → 23
3. e(15) + 23 → 38
4. c(27) + f(35) → 62
5. 38 + 62 → 100 (root)

(b) Huffman Tree and Codes [2 marks]

Smaller-weight child on the left (edge 0), larger-weight child on the right (edge 1).



Symbol	Frequency	Code	Length
a	8	0101	4
b	3	0100	4
c	27	10	2
d	12	011	3
e	15	00	2
f	35	11	2

(c) Total Bits Comparison [1 mark]

Huffman: 8(4) + 3(4) + 27(2) + 12(3) + 15(2) + 35(2) = 32 + 12 + 54 + 36 + 30 + 70 = **234 bits**

Fixed-length (3 bits): 100 × 3 = **300 bits**

Savings: 300 − 234 = **66 bits**

Question 2: Counting Sort (5 marks)

(a) Scanning from Left to Right [2 marks]

If the placement phase scans left to right, equal-key elements come out in reverse relative order, so the sort **loses stability**.

Why: Each new equal-key element is placed at the end of its block, so earlier elements end up behind later ones.

Why it matters: Counting sort is used as the subroutine in Radix sort, which depends on stability to preserve the order from previous digit passes.

(b) Estimate the Number of Operations [2 marks]

Given: n = 1000, keys in range 0..10

- **Counting sort:** O(n + k) = 1000 + 10 ≈ 1,000,001,000 operations
- **Comparison sort:** n log n = 1000 × log (1000) ≈ 1000 × 10 = 10,000 operations

Conclusion: Comparison sort is much better here because the key range is huge compared with the number of elements.

(c) Sorting Integers in the Range −50..50 [1 mark]

Use an offset of 50 to shift the range [−50, 50] to [0, 100]: index = x + 50.

Use a count array of size 101, apply counting sort, then subtract 50 from the sorted values to restore the original range.